claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a3b64b60599afebae.jsonl`
- SHA-256: `289fafac678899f72c2ee4f7ea79d6d971322e49ff15090403fc09befce51a75`
- Source modified: `2026-06-10T16:29:31+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:22:08.063Z)

```
You are an adversarial verifier. A code reviewer audited the repo at
C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: `db backup` deletes the destination before any validation ? `muneem db backup <ledger-
path>` destroys the ledger
SEVERITY CLAIMED: high
FILE: src/muneem/db.py (line 292-295)
DESCRIPTION: `backup_database` unlinks `dest` BEFORE opening the source and never checks that
dest != source. If the user passes the ledger's own path as the destination (an easy slip ? `db
backup` takes the destination as its positional argument, and a user may think they are naming
the source), the live encrypted ledger is deleted; `_open_encrypted` then creates a brand-new
empty encrypted DB at that path (key verification trivially passes on an empty DB), and the
'backup' completes successfully, leaving an empty ledger. Total silent data loss with exit code
0. There is also no overwrite confirmation for an existing backup file (cli.py:450-465 has no
--force / prompt).
EVIDENCE QUOTED: dest_path = Path(dest)
    dest_path.parent.mkdir(parents=True, exist_ok=True)
    dest_path.unlink(missing_ok=True)
    source = _open_encrypted(str(db_path), key)  # validates key
SUGGESTED FIX: Resolve both paths and refuse if `dest_path.resolve() == Path(db_path).resolve()`
(also reject dest equal to the source's -wal/-shm siblings). Open and validate the source BEFORE
unlinking dest. In the CLI, require confirmation (or --force) when dest already exists.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario
can actually occur in realistic single-user use on Windows, and whether the assigned severity is
justified.
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot
confirm the problem is real, set isReal=false. If it is real but over/under-stated, set
adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,
testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL
financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file
existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess
sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers,
balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account
number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.
```

2. user (2026-06-10T16:22:13.125Z)

```
      """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.

      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
      (accounts, transactions, holdings, merchants, items) grows with later milestones.
      ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
      """

      from __future__ import annotations

      import os
      import sqlite3
      from collections.abc import Iterator
      from contextlib import contextmanager
      from pathlib import Path

      from muneem.errors import EncryptedDatabaseError  # re-exported so
db.EncryptedDatabaseError works

      SCHEMA_VERSION = 4

      # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
      # file does not, so we can tell them apart without a key.
      _SQLITE_MAGIC = b"SQLite format 3\x00"

      _SCHEMA = """
      CREATE TABLE IF NOT EXISTS documents (
          id            INTEGER PRIMARY KEY,
          source        TEXT NOT NULL,
          external_id   TEXT,
          sender        TEXT,
          subject       TEXT,
          received_date TEXT,
          doc_type      TEXT,
          sha256        TEXT NOT NULL UNIQUE,
          path          TEXT,
          status        TEXT NOT NULL DEFAULT 'parsed',
          created_at    TEXT NOT NULL DEFAULT (datetime('now'))
      );

      CREATE TABLE IF NOT EXISTS offers (
          id          INTEGER PRIMARY KEY,
          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
          merchant    TEXT,
          promo_code  TEXT,
          description TEXT,
          expiry_raw  TEXT,
          expiry_type TEXT,
          source      TEXT NOT NULL DEFAULT 'text',
          redeemed    INTEGER NOT NULL DEFAULT 0,
          source_text TEXT
      );

      CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);

      -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
      -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
      -- Normalization into a unified view layer is planned for a later phase
      -- once all bank schemas are stabilized.
      CREATE TABLE IF NOT EXISTS axis_transactions (
          id INTEGER PRIMARY KEY,
          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
          account_number TEXT,
          txn_date TEXT,
```

```
        value_date TEXT,
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );

    CREATE TABLE IF NOT EXISTS axis_cc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        particulars TEXT,
        merchant_category TEXT,
        amount REAL,
        type TEXT
    );

    CREATE TABLE IF NOT EXISTS au_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );

    CREATE TABLE IF NOT EXISTS hdfc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        ref TEXT,
        withdrawal REAL,
        deposit REAL,
        closing_balance REAL
    );
    """


    def _raw_key_pragma(key: str) -> str:
        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
        return f"PRAGMA key = \"x'{key}'\""


    def is_plaintext_sqlite(db_path: Path | str) -> bool:
        """True iff the file exists and begins with the *unencrypted* SQLite magic header.

        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
        without needing the key.
        """
        try:
            with open(db_path, "rb") as fh:
                return fh.read(16) == _SQLITE_MAGIC
        except OSError:
            return False


    def _harden_permissions(path: str) -> None:
```

```
        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
        if path == ":memory:":
            return
        try:
            os.chmod(path, 0o600)
        except OSError:
            pass


    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
        """Open an encrypted connection via SQLCipher, verifying the key before returning.

        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
        key is wrong (or the file is not an encrypted muneem ledger), this raises.
        """
        try:
            import sqlcipher3
        except ImportError as exc:  # pragma: no cover - exercised only without the binding
            raise EncryptedDatabaseError(
                "database encryption requested but the 'sqlcipher3' package is not
installed"
            ) from exc

        conn = sqlcipher3.connect(path)
        conn.execute(_raw_key_pragma(key))  # MUST be the first statement on the connection
        try:
            conn.execute("SELECT count(*) FROM sqlite_master")  # forces key verification
        except sqlcipher3.DatabaseError as exc:
            conn.close()
            raise EncryptedDatabaseError(
                f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
                "encrypted muneem ledger"
            ) from exc
        conn.row_factory = sqlcipher3.Row
        return conn


    @contextmanager
    def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
        """Open a connection (commit on success, always close). Parent dirs created.

        With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
        SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
        ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
        :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
        """
        path = str(db_path)
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        if key is not None:
            conn = _open_encrypted(path, key)
            _harden_permissions(path)
        else:
            conn = sqlite3.connect(path)
            conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
```

```
        finally:
            conn.close()


    @contextmanager
    def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
        """Open the real on-disk ledger, encrypted with the keychain-managed key.

        The production path: resolves (or mints, on first use) the 256-bit key from the OS
        keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
        file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
        "file is not a database" error.
        """
        from muneem.db_key import get_or_create_key

        path = str(db_path)
        if is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
                "to migrate it to an encrypted ledger"
            )
        with connect(path, key=get_or_create_key()) as conn:
            yield conn


    def encrypt_database(db_path: Path | str, key: str) -> int:
        """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.

        The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
        securely delete (it still holds the cleartext data). Returns the number of user
tables
        copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
        """
        import sqlcipher3

        path = Path(db_path)
        if not is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
            )

        enc_tmp = path.with_name(path.name + ".enc.tmp")
        enc_tmp.unlink(missing_ok=True)
        try:
            src = sqlcipher3.connect(str(path))  # open the plaintext source (no key)
            try:
                src_tables = src.execute(
                    "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
                ).fetchone()[0]
                target = str(enc_tmp).replace("\\", "/")  # SQL string literal: forward
slashes
                src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"x'{key}'\"")
                src.execute("SELECT sqlcipher_export('encrypted')")
                src.execute("DETACH DATABASE encrypted")
            finally:
                src.close()

            # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
```

```
            # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
            # so a failed check is recoverable ? but we refuse to swap in a bad file.)
            with connect(str(enc_tmp), key=key) as check:
                integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
                ntables = check.execute(
                    "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
                ).fetchone()[0]
            if integrity != "ok" or ntables != src_tables:
                raise EncryptedDatabaseError(
                    f"encryption migration verification failed for {path.name} "
                    f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
                )

            backup = path.with_name(path.name + ".plaintext.bak")
            backup.unlink(missing_ok=True)
            path.rename(backup)
            enc_tmp.rename(path)  # consumed on success
            _harden_permissions(str(path))
        finally:
            # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
            enc_tmp.unlink(missing_ok=True)
        return int(ntables)


    def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
        """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).

        Opens with ``current_key`` first (which verifies it), then rekeys. Raises
        :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
        muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
        after this returns successfully.
        """
        conn = _open_encrypted(str(db_path), current_key)  # validates current_key
        try:
            conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
            conn.commit()
        finally:
            conn.close()
        _harden_permissions(str(db_path))


    def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
        """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).

        Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
        naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
        it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
        """
        import sqlcipher3

        dest_path = Path(dest)
        dest_path.parent.mkdir(parents=True, exist_ok=True)
        dest_path.unlink(missing_ok=True)
        source = _open_encrypted(str(db_path), key)  # validates key
        try:
            target = sqlcipher3.connect(str(dest_path))
            try:
```

```
                target.execute(_raw_key_pragma(key))  # key the destination so the backup is
encrypted
                source.backup(target)  # consistent snapshot of the live DB
            finally:
                target.close()
        finally:
            source.close()
        _harden_permissions(str(dest_path))


    def init_db(conn: sqlite3.Connection) -> None:
        """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).

        The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
        are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
        """
        from muneem import migrations

        migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)


    def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
        return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()


    def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
        """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
        cols = ", ".join(fields)
        placeholders = ", ".join("?" for _ in fields)
        cur = conn.execute(
            f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
            tuple(fields.values()),
        )
        rowid = cur.lastrowid  # always set after a successful INSERT; guard for the type-
checker
        if rowid is None:  # pragma: no cover - defensive; INSERT always sets lastrowid
            raise RuntimeError(f"INSERT into {table} did not return a row id")
        return int(rowid)


    def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
        """Insert into documents table. Delegates to the shared helper for DRY."""
        return _insert_row(conn, "documents", dict(fields))


    def record_provenance(
        conn: sqlite3.Connection,
        *,
        document_id: int,
        source_type: str,
        parser: str | None = None,
        parser_version: str | None = None,
        mapping_layer: str | None = None,
        confidence: str | None = None,
    ) -> None:
        """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).

        First-class but *not* per transaction row: every txn table FKs ``documents(id)``, so
a single
```

```
        provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
        overwrites rather than duplicating).
        """
        conn.execute(
            "INSERT INTO provenance "
            "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
            "VALUES (?, ?, ?, ?, ?, ?) "
            "ON CONFLICT(document_id) DO UPDATE SET "
            "source_type=excluded.source_type, parser=excluded.parser, "
            "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
            "confidence=excluded.confidence",
            (document_id, source_type, parser, parser_version, mapping_layer, confidence),
        )


    def record_parse(
        conn: sqlite3.Connection,
        document_id: int,
        *,
        confidence: str,
        parser: str,
        source_type: str = "pdf_statement",
        parser_version: str | None = None,
        mapping_layer: str = "deterministic",
    ) -> None:
        """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
        conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
        record_provenance(
            conn,
            document_id=document_id,
            source_type=source_type,
            parser=parser,
            parser_version=parser_version,
            mapping_layer=mapping_layer,
            confidence=confidence,
        )


    def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
        return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()


    def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "offers", fields)


    def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
        return conn.execute(
            """
            SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
                   d.external_id, d.path
            FROM offers o
            JOIN documents d ON d.id = o.document_id
            ORDER BY o.id
            """
        ).fetchall()


    def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_transactions", fields)

```

```
416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451
```

3. user (2026-06-10T16:22:14.463Z)

```
398-            f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399-        )
400-        return
401-    if not dbmod.is_plaintext_sqlite(db_path):
402-        typer.echo(
403-            f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing to
do."
404-        )
405-        return
406-    n = dbmod.encrypt_database(db_path, get_or_create_key())
407-    typer.echo(
408:        f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup remains
at "
409-        f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410-    )
411-
412-
413-def _require_encrypted_ledger(db_path: Path) -> str:
414-    """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415-    from muneem import db as dbmod
```

```
416-    from muneem.db_key import peek_key
417-
418-    if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
--
440-    db_key.set_key(fresh)
441-    try:
442-        dbmod.rekey_database(db_path, current, fresh)
443-    except dbmod.EncryptedDatabaseError as exc:
444-        db_key.set_key(current)  # roll back; the file is still under the old key
445-        typer.echo(str(exc), err=True)
446-        raise typer.Exit(1) from exc
447-    typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448-
449-
450:@db_app.command("backup")
451:def db_backup(
452:    dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453-    db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454-) -> None:
455:    """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456-    from muneem import db as dbmod
457-
458-    db_path = db or load_config().db_path
459-    key = _require_encrypted_ledger(db_path)
460-    try:
461:        dbmod.backup_database(db_path, dest, key)
462-    except dbmod.EncryptedDatabaseError as exc:
463-        typer.echo(str(exc), err=True)
464-        raise typer.Exit(1) from exc
465-    typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466-
467-
468-@db_app.command("setup-recovery")
469-def db_setup_recovery(
470-    out: Path | None = typer.Option(
471-        None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
472-    ),
473-) -> None:
474-    """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475-
476:    Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477-    file holds only the *wrapped* key ? useless without the passphrase.
478-    """
479-    import getpass
480-
481-    from muneem import recovery
482-    from muneem.db_key import peek_key
483-
484-    key = peek_key()
485-    if not key:
486-        typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
--
491-        typer.echo("passphrases did not match.", err=True)
492-        raise typer.Exit(1)
493-    try:
494-        recovery.write_recovery_file(out_path, key, passphrase)
495-    except recovery.RecoveryError as exc:
496-        typer.echo(str(exc), err=True)
497-        raise typer.Exit(1) from exc
498-    typer.echo(
```

```
499-        f"wrote recovery file to {out_path}. Store it OFF this machine (password manager / "
500-        "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501:        "lost keychain means the ledger and its backups are unrecoverable."
502-    )
503-
504-
505-@db_app.command("recover")
506-def db_recover(
507-    file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
508-    db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
509-) -> None:
510-    """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
511-    import getpass
```

4. user (2026-06-10T16:22:58.954Z)

```
No files found
```

5. user (2026-06-10T16:23:00.329Z)

```
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-54-
tests\test_db_crypto_ops.py-55-def test_backup_is_encrypted_and_intact(tmp_path):
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-56-    p = tmp_path / "led.db"
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-57-    _seed_encrypted(p, KEY_A, n=4)
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-58-    dest = tmp_path / "backup" /
"led.bak.db"
tests\test_db_crypto_ops.py:59:    dbmod.backup_database(p, dest, KEY_A)
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-60-    assert dest.exists() and not
dbmod.is_plaintext_sqlite(dest)  # backup stays encrypted
tests\test_db_crypto_ops.py-61-    with dbmod.connect(dest, key=KEY_A) as conn:
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-62-        assert
conn.execute("SELECT count(*) FROM t").fetchone()[0] == 4
tests\test_db_crypto_ops.py-63-    with pytest.raises(dbmod.EncryptedDatabaseError):  # wrong
key can't read the backup
tests\test_db_crypto_ops.py-64-        with dbmod.connect(dest, key=KEY_B):
--
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-67-
tests\test_db_crypto_ops.py-68-def test_backup_with_wrong_key_raises(tmp_path):
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-69-    p = tmp_path / "led.db"
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-70-    _seed_encrypted(p, KEY_A)
tests\test_db_crypto_ops.py-71-    with pytest.raises(dbmod.EncryptedDatabaseError):
tests\test_db_crypto_ops.py:72:        dbmod.backup_database(p, tmp_path / "b.db", KEY_B)
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-73-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-74-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-75-# --- migration cleanup +
correctness at size + plaintext refusal -----------------------------
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-76-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-77-
--
tests\test_db_crypto_ops.py-113-        with dbmod.connect_encrypted(p):
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-114-            pass
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-115-    assert "muneem db encrypt" in
str(ei.value)  # actionable guidance, not a cryptic error
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-116-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-117-
tests\test_db_crypto_ops.py:118:# --- CLI: db rotate-key / db backup
------------------------------------------------------------
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-119-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-120-
tests\test_db_crypto_ops.py-121-def _new_encrypted_ledger(path):
tests\test_db_crypto_ops.py-122-    with dbmod.connect_encrypted(path) as conn:
```

```
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-123-        dbmod.init_db(conn)
```

6. user (2026-06-10T16:23:00.603Z)

```
.venv\Scripts\python.exe
```

7. user (2026-06-10T16:23:09.361Z)

```
1       """Crypto operations on the encrypted ledger: key rotation, encrypted backup, migration
temp
2       cleanup, correctness at size, and the plaintext-refusal guidance ? plus their CLI
commands.
3
4       Offline, fabricated data only; the autouse conftest fixture keeps the real keychain
untouched.
5       """
6
7       import sqlite3
8
9       import pytest
10
11      pytest.importorskip("sqlcipher3")
12
13      from typer.testing import CliRunner  # noqa: E402
14
15      from muneem import db as dbmod  # noqa: E402
16      from muneem import db_key  # noqa: E402
17      from muneem.cli import app  # noqa: E402
18
19      runner = CliRunner()
20
21      KEY_A = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
22      KEY_B = "ffeeddccbbaa99887766554433221100ffeeddccbbaa99887766554433221100"
23
24
25      def _seed_encrypted(path, key, n=3):
26          with dbmod.connect(path, key=key) as conn:
27              conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
28              conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(n)])
29
30
31      # --- rekey / rotation
------------------------------------------------------------------------
32
33
34      def test_rekey_changes_the_key_and_keeps_data(tmp_path):
35          p = tmp_path / "led.db"
36          _seed_encrypted(p, KEY_A, n=5)
37          dbmod.rekey_database(p, KEY_A, KEY_B)
38          with pytest.raises(dbmod.EncryptedDatabaseError):  # old key no longer opens it
39              with dbmod.connect(p, key=KEY_A):
40                  pass
41          with dbmod.connect(p, key=KEY_B) as conn:  # new key works, data intact
42              assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5
43
44
45      def test_rekey_with_wrong_current_key_raises(tmp_path):
46          p = tmp_path / "led.db"
47          _seed_encrypted(p, KEY_A)
48          with pytest.raises(dbmod.EncryptedDatabaseError):
49              dbmod.rekey_database(p, KEY_B, KEY_A)
50
51
52      # --- encrypted backup
```

```
------------------------------------------------------------------------
53
54
55      def test_backup_is_encrypted_and_intact(tmp_path):
```

8. user (2026-06-10T16:25:22.294Z)

```
Command running in background with ID: bs2y7fao9. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\bs2y7fao9.output. You will be notified when it
completes. To check interim output, use Read on that file path.
```

9. user (2026-06-10T16:25:30.744Z)

```
<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file
has 1 lines.</system-reminder>
```

10. user (2026-06-10T16:25:44.096Z)

```
(Bash completed with no output)
```

11. user (2026-06-10T16:26:01.937Z)

```
---exit---
```

12. user (2026-06-10T16:26:34.310Z)

```
0 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
muneem/02b9e9c9-3806-45ca-9b24-2ecc35a81efd/tasks/bs2y7fao9.output
```

13. user (2026-06-10T16:26:53.728Z)

```
   Id StartTime            ProcessName
   -- ---------            -----------
 6708 6/10/2026 9:53:34 PM python
15232 6/10/2026 9:54:05 PM python
18780 6/10/2026 9:54:05 PM python
27340 6/10/2026 9:53:21 PM python
31424 6/10/2026 9:53:34 PM python
36308 6/10/2026 9:53:21 PM python
```

14. assistant (2026-06-10T16:27:13.857Z)

```
import tempfile, threading, sys
from pathlib import Path
from muneem import db as dbmod

KEY = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"

tmp = Path(tempfile.mkdtemp())
p = tmp / "ledger.db"

with dbmod.connect(p, key=KEY) as conn:
    conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
    conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in range(5)])

print("before: size =", p.stat().st_size, flush=True)

result = {"state": "did not finish"}

def run():
    try:
        dbmod.backup_database(p, p, KEY)  # dest == source
        result["state"] = "returned without raising (CLI would print success, exit 0)"
```

```
    except Exception as e:
        result["state"] = f"raised {type(e).__name__}: {e}"

t = threading.Thread(target=run, daemon=True)
t.start()
t.join(timeout=15)
if t.is_alive():
    print("backup_database HUNG (>15s, still running)", flush=True)
else:
    print("backup_database:", result["state"], flush=True)

print("after: exists =", p.exists(), "size =", (p.stat().st_size if p.exists() else None),
flush=True)
for sib in (".db-wal", ".db-shm"):
    q = p.with_suffix(sib)
    if q.exists():
        print("sibling:", q.name, q.stat().st_size, flush=True)
```

Can we still read the original data? (use a fresh path-string connection)

```
try:
    import sqlcipher3
    c = sqlcipher3.connect(str(p))
    c.execute(f"PRAGMA key = \"x'{KEY}'\"")
    tables = c.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall()
    print("tables now:", [r[0] for r in tables], flush=True)
    if any(r[0] == "t" for r in tables):
        print("rows in t:", c.execute("SELECT count(*) FROM t").fetchone()[0], flush=True)
    c.close()
except Exception as e:
    print("reopen failed:", type(e).__name__, "-", e, flush=True)

sys.exit(0)
```

15. user (2026-06-10T16:27:13.880Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\muneem_verify_backup_self.py
(file state is current in your context ? no need to Read it back)
```

16. user (2026-06-10T16:27:39.181Z)

```
before: size = 8192
backup_database HUNG (>15s, still running)
after: exists = True size = 0
tables now: []
```

17. user (2026-06-10T16:28:19.402Z)

```
ProcessId   : 36308
CommandLine : "C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe" -

ProcessId   : 27340
CommandLine : "C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe" -

ProcessId   : 24288
CommandLine : C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe -u
              C:/Users/avidu/AppData/Local/Temp/muneem_backup_repro.py

ProcessId   : 12344
CommandLine : C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe -u
              C:/Users/avidu/AppData/Local/Temp/muneem_backup_repro.py
```

18. user (2026-06-10T16:28:35.537Z)

killed hung stdin repros

19. assistant (2026-06-10T16:28:46.273Z)

```
"""Secondary check: unlink-before-validate destroys an EXISTING good backup at dest
when the source key is wrong (i.e., a routine re-backup that fails still deletes the
previous good backup)."""
import tempfile
from pathlib import Path
from muneem import db as dbmod

KEY_A = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
KEY_B = "ffeeddccbbaa99887766554433221100ffeeddccbbaa99887766554433221100"

tmp = Path(tempfile.mkdtemp())
ledger = tmp / "ledger.db"
dest = tmp / "good_old_backup.db"

with dbmod.connect(ledger, key=KEY_A) as conn:
    conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY)")
    conn.execute("INSERT INTO t DEFAULT VALUES")

# make a legit prior backup at dest
dbmod.backup_database(ledger, dest, KEY_A)
print("prior backup exists, size =", dest.stat().st_size, flush=True)

# now a backup attempt with the WRONG key (raises, per existing test)
try:
    dbmod.backup_database(ledger, dest, KEY_B)
except Exception as e:
    print("raised as expected:", type(e).__name__, flush=True)

print("after failed attempt: dest exists =", dest.exists(), flush=True)
```

20. user (2026-06-10T16:28:50.274Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\muneem_verify_wrongkey_backup.py
(file state is current in your context ? no need to Read it back)
```

21. user (2026-06-10T16:28:59.545Z)

```
prior backup exists, size = 8192
2■0■2■6■-■0■6■-■1■0■ ■2■1■:■5■8■:■5■9■.■3■8■0■:■ ■E■R■R■O■R■ ■C■O■R■E■
■s■q■l■c■i■p■h■e■r■_■p■a■g■e■_■c■i■p■h■e■r■:■ ■h■m■a■c■ ■c■h■e■c■k■ ■f■a■i■l■e■d■ ■f■o■r■
■p■g■n■o■=■1■
■2■0■2■6■-■0■6■-■1■0■ ■2■1■:■5■8■:■5■9■.■3■8■2■:■ ■E■R■R■O■R■ ■C■O■R■E■
■s■q■l■i■t■e■3■C■o■d■e■c■:■ ■e■r■r■o■r■ ■d■e■c■r■y■p■t■i■n■g■ ■p■a■g■e■ ■1■ ■d■a■t■a■:■ ■1■
■2■0■2■6■-■0■6■-■1■0■ ■2■1■:■5■8■:■5■9■.■3■8■2■:■ ■E■R■R■O■R■ ■C■O■R■E■
■s■q■l■c■i■p■h■e■r■_■c■o■d■e■c■_■c■t■x■_■s■e■t■_■e■r■r■o■r■ ■1■
■raised as expected: EncryptedDatabaseError
after failed attempt: dest exists = False
```

22. user (2026-06-10T16:29:25.395Z)

```
Structured output provided successfully
```

23. assistant (2026-06-10T16:29:31.431Z)

```
Verification complete.
```